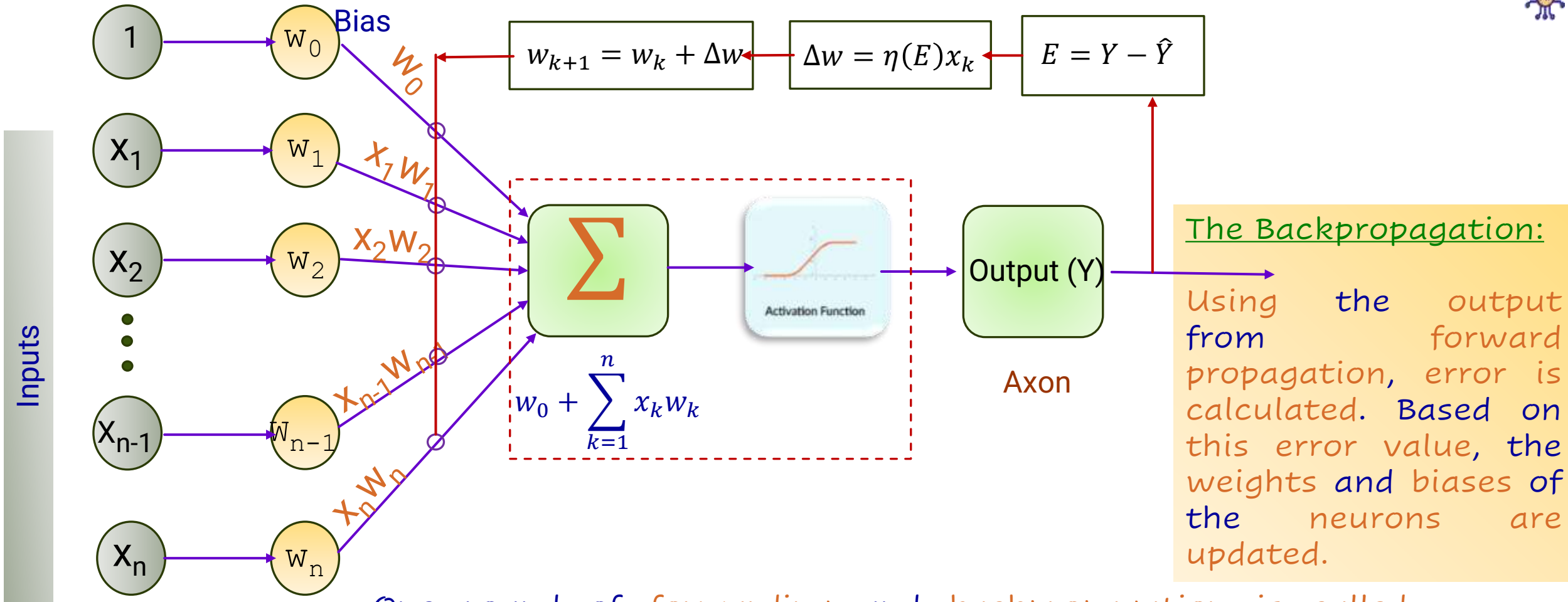
Back propagation and Epoch



The Backpropagation:

Using the output from forward propagation, error is calculated. Based on this error value, the weights and biases of the neurons are updated.

One round of forwarding and backpropagation is called iteration. Total number of iteration for training set aka "Epoch".

Mean Squared Error (MSE):

Mean Squared Error :
Measure how the model is performing across all the training data.

$$J = \frac{1}{n}\sum_{i=1}^{n}(y_i - y_i')^2$$

J is the Mean Squared Error, there are n features in the training data

y_i is the actual value of the i^{th} feature in the training data

y_i' is the predicted value of the i^{th} feature in the trainng data

Once we've measured the MSE for a model, we adjust the weights in the model and repeat, until the model converges

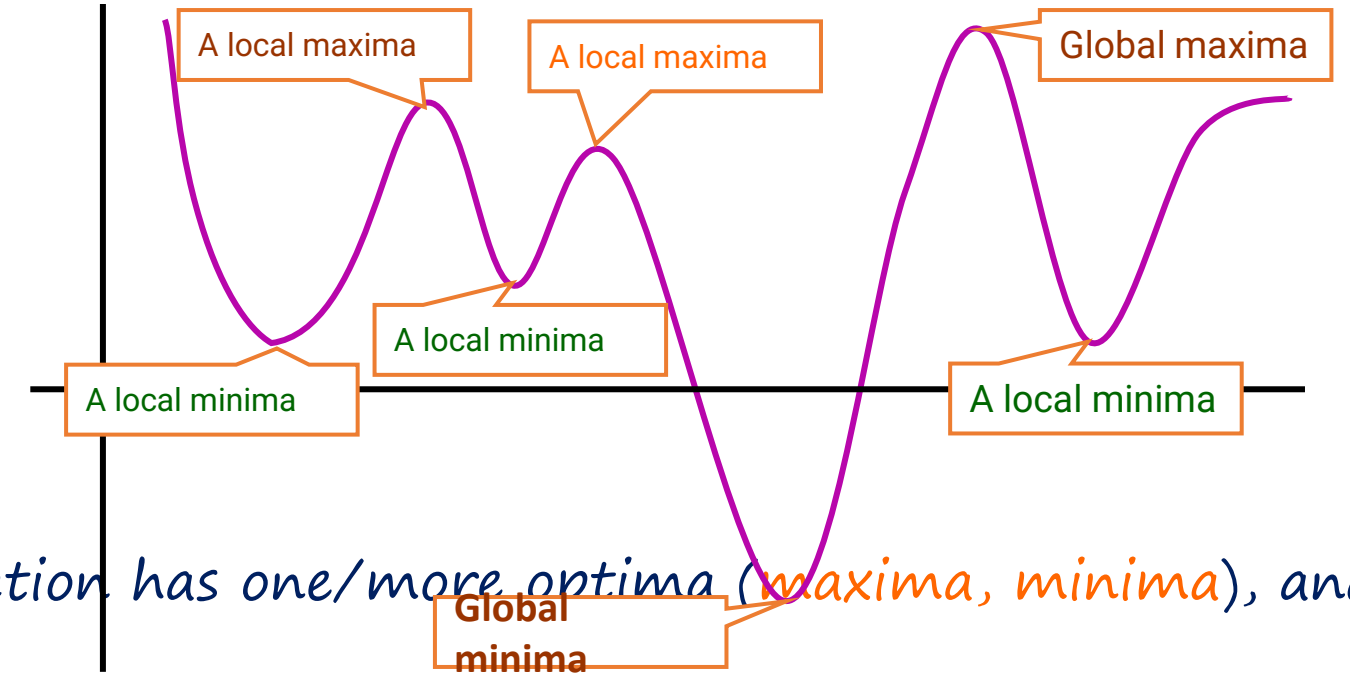
Mean Squared Error stops changing or only changes by a very small amount.

Functions and Optimizations

The objective function of the ML problem we are solving (e.g., squared loss for regression)

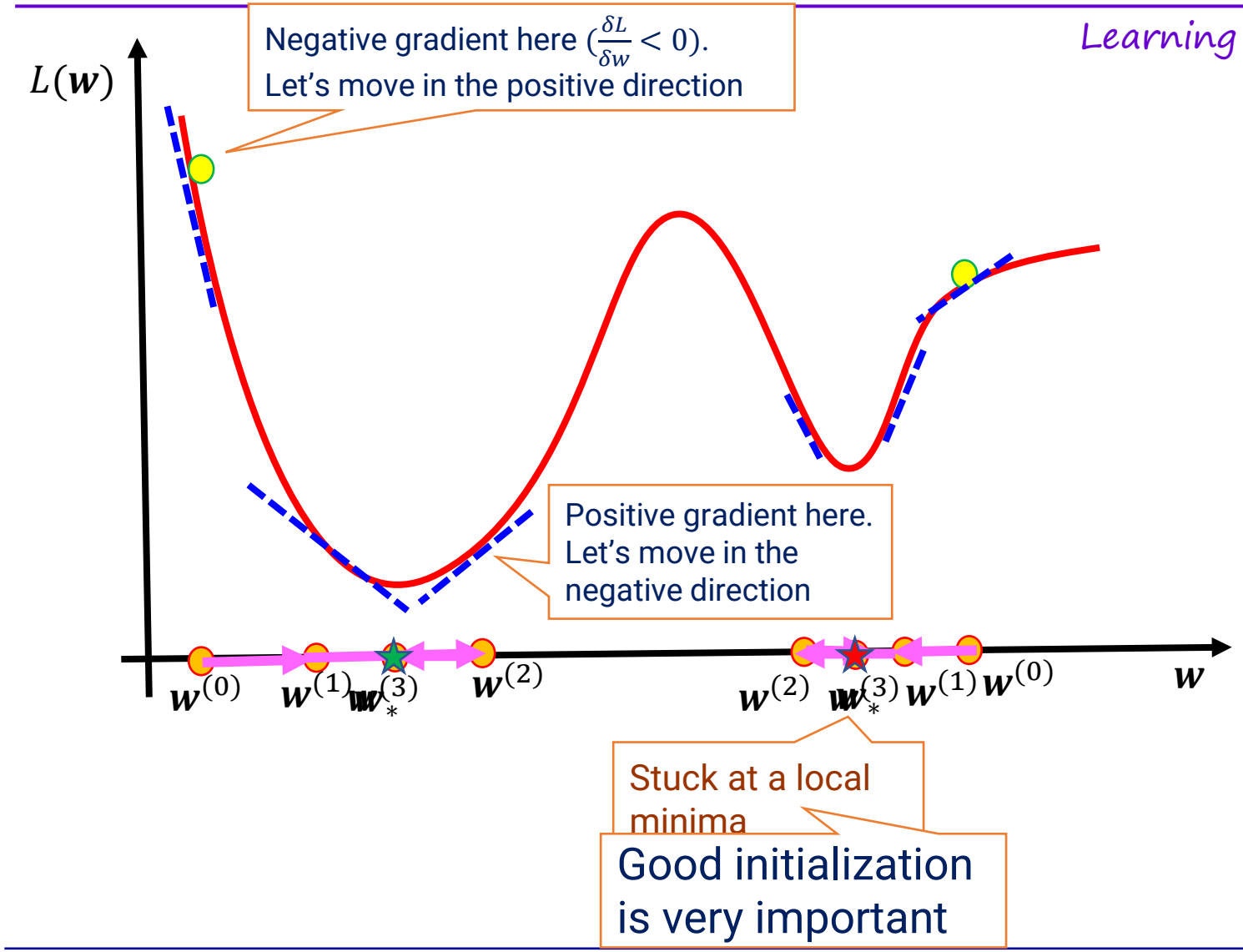
Assume unconstrained for now, i.e., just a real-valued number/vector

- Many ML problems require us to optimize a function f of some variable(s) x
- For simplicity, assume f is a scalar-valued function of a scalar $x (f: \mathbb{R} \rightarrow \mathbb{R})$

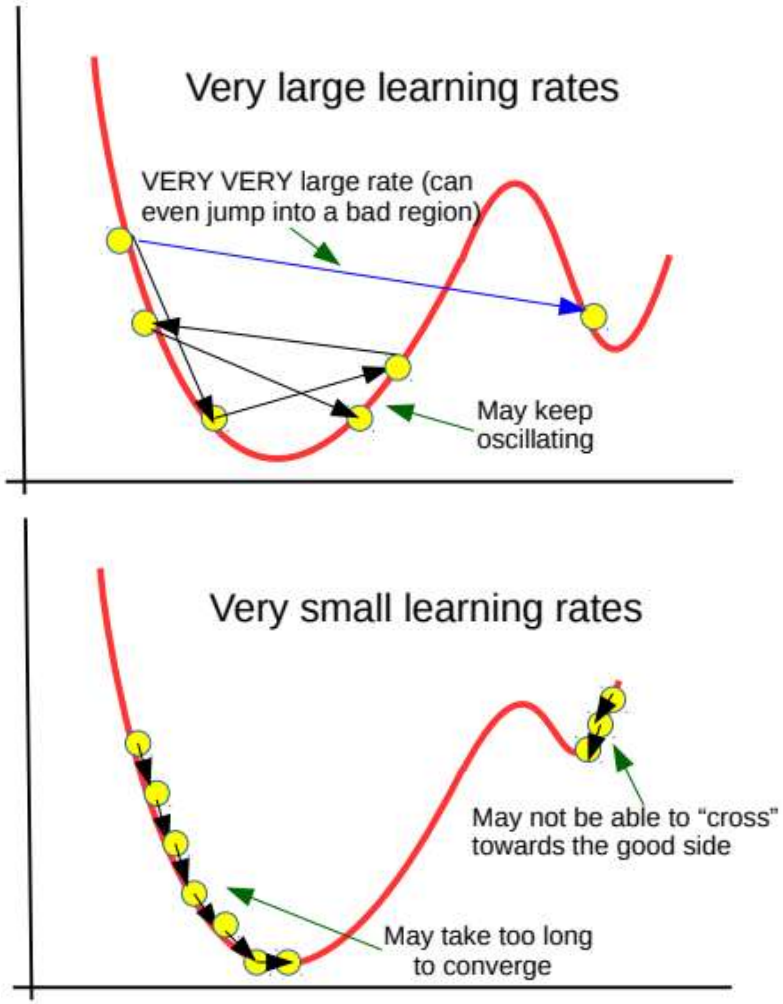


- Any function has one/more optima (maxima, minima), and maybe saddle points
- Finding the optima or saddles requires derivatives/gradients of the function

GD: an Illustration



Learning rate/Gradient is very important



Training dataset too large to be feed in computer at once.

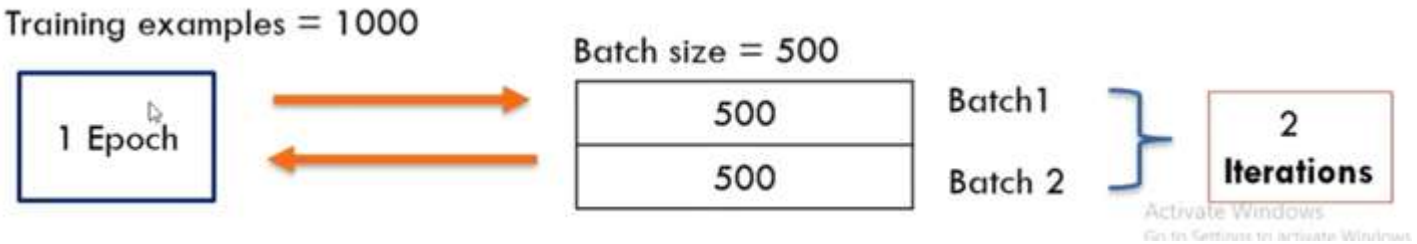
Batch, Batch size, and Iterations

Batch: To divide the training dataset into multiple smaller trainingsets.

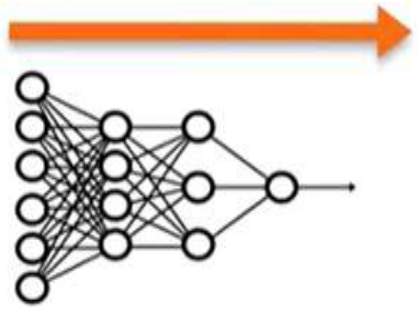
Batch size: Total number of training examples present in a single batch.

Batch size is a hyperparameter.

Iterations: Iterations are the number of batches needed to complete one epoch.



Epoch: One entire passing of training data through the model



1 Epoch

2 Epoch

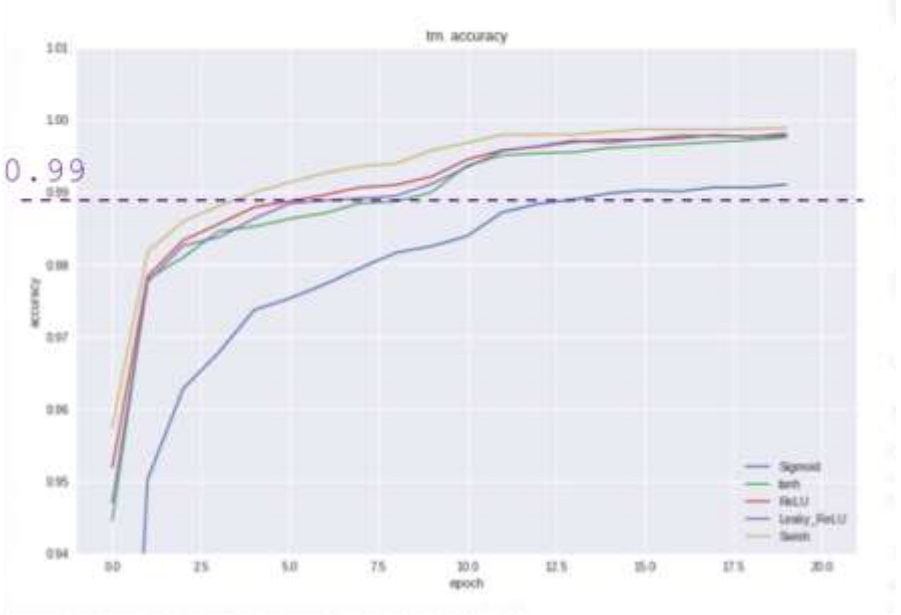
3 Epoch

N Epoch

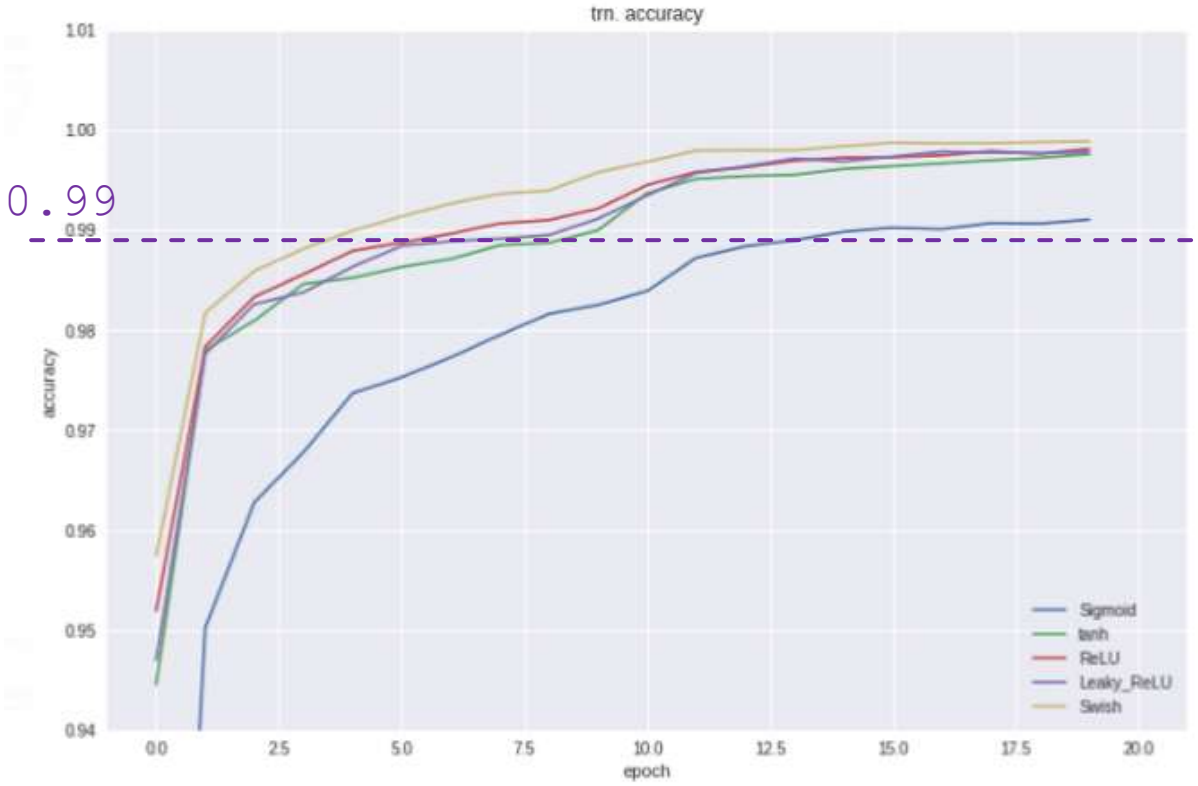
The weight will be decent

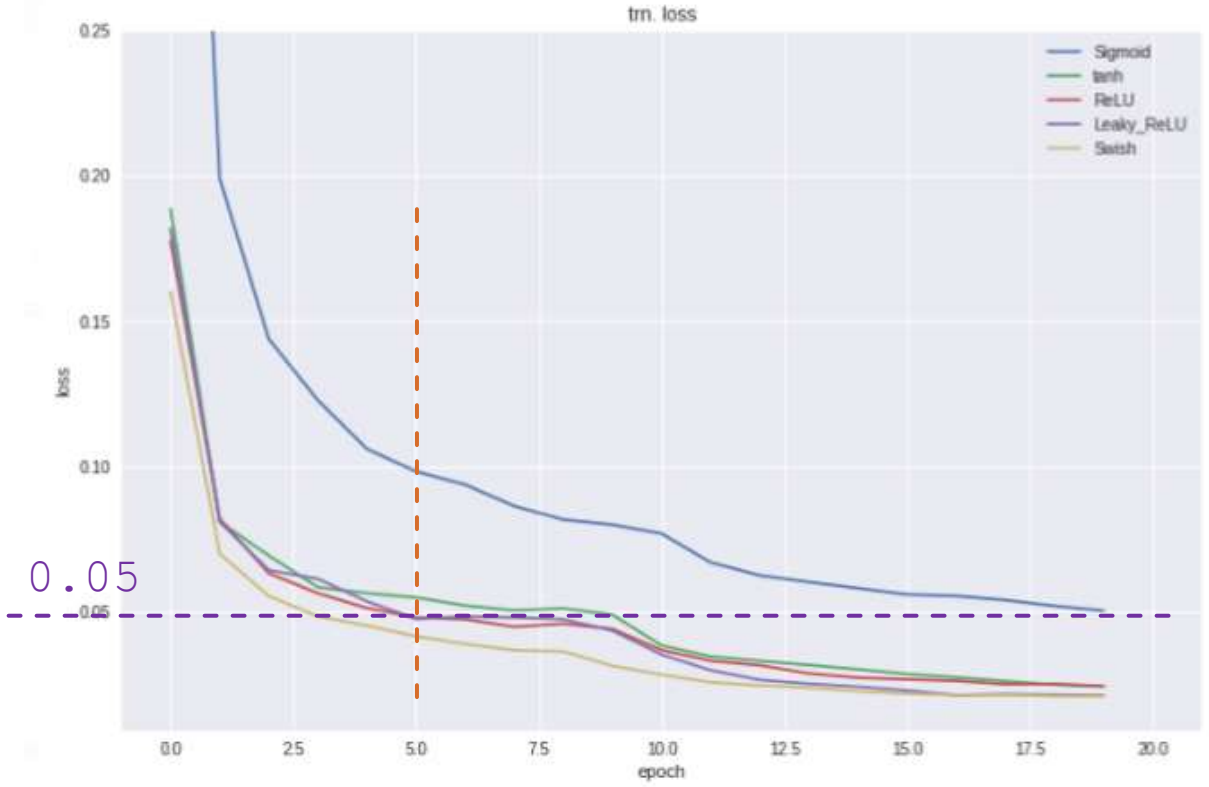
The weight will be improved

The number of epochs is a hyperparameter that defines the number of times that the learning algorithm will work through the entire training dataset.



Model accuracy and loss Curve: Various activation function (Epoch)



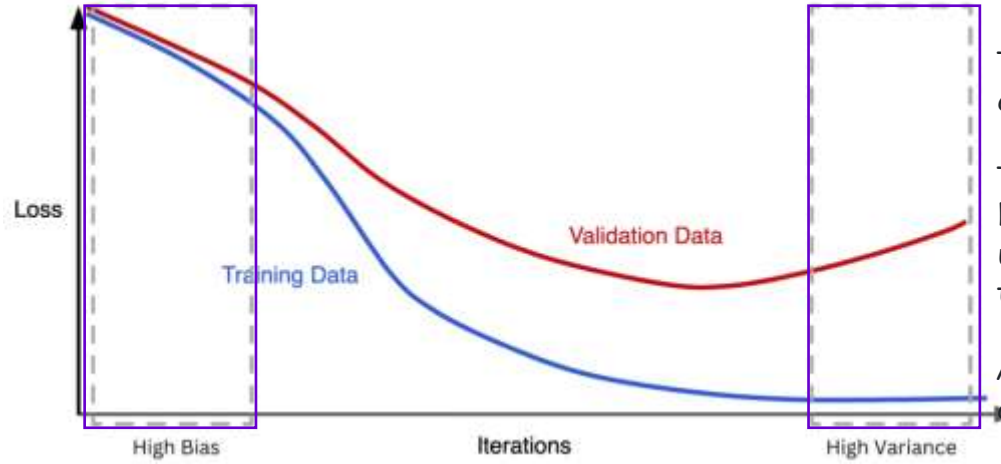


Loss Curve w.r.t Epoch: Training and Validation dataset

The model is too simple to capture the data's complexity.

This results in high training and validation losses that are very similar, indicating underfitting.

The overall performance remains poor because the model is unable to learn the underlying patterns in the data.



The validation loss often starts to rise after a certain point.

This trend indicates that the model is becoming too complex, capturing not only the underlying patterns but also the noise in the training data.

As a result, it struggles to generalize to new data

Model Bias and Variance

Bias

Bias is defined as error occurring between the model's predicted value and the actual value.

Bias is a systematic error that occurs due to wrong assumptions

Variance

Variance is the variability of model prediction for a given data point or a value.

It tells us spread of our data assumptions are taken to build the target function.

In this case, the model will not match the training dataset closely.

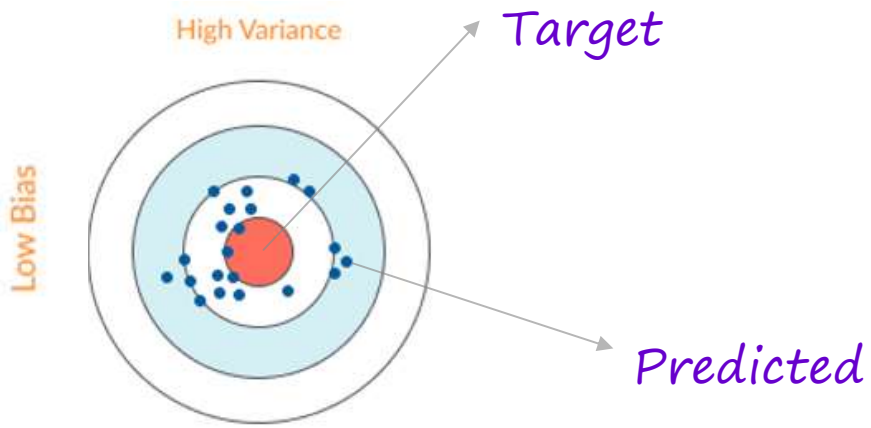
Overfitting

Overfitting happens when the neural network is very good at learning its training set, but cannot generalize beyond the training set (known as the generalization problem

The symptoms of Overfitting are:

Low bias – accurate predictions for the training set

High variance – poor ability to generate predictions for the validation set



Drop-Out in Deep Learning NN

Hinton speaking at the Nobel Prize Lectures
Stockholm, 2024

Geoffrey Everest Hinton (born 6 December 1947) is a British-Canadian computer scientist, cognitive scientist, and cognitive psychologist known for his work on artificial neural networks, which earned him the title "the Godfather of AI".[8]

Hinton is University Professor Emeritus at the University of Toronto. From 2013 to 2023, he divided his time working for Google (Google Brain) and the University of Toronto before publicly announcing his departure from Google in May 2023, citing concerns about the many risks of artificial intelligence (AI) technology.[9][10] In 2017, he co-founded and became the chief scientific advisor of the Vector Institute in Toronto.[11][12]

Journal of Machine Learning Research 15 (2014) 1929-1958 Submitted 11/

Dropout: A Simple Way to Prevent Neural Networks from Overfitting

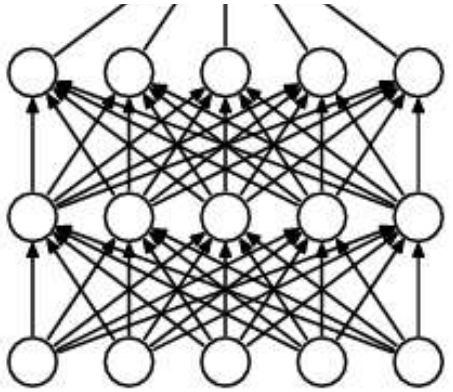
Nitish Srivastava NITISH@CS.TORONTO.EDU
Geoffrey Hinton HINTON@CS.TORONTO.EDU
Alex Krizhevsky KRIZ@CS.TORONTO.EDU
Ilya Sutskever ILYA@CS.TORONTO.EDU
Ruslan Salakhutdinov RSALAKHU@CS.TORONTO.EDU
Department of Computer Science
University of Toronto
10 Kings College Road, Rm 3302
Toronto, Ontario, M5S 3G4, Canada.

Editor: Yoshua Bengio

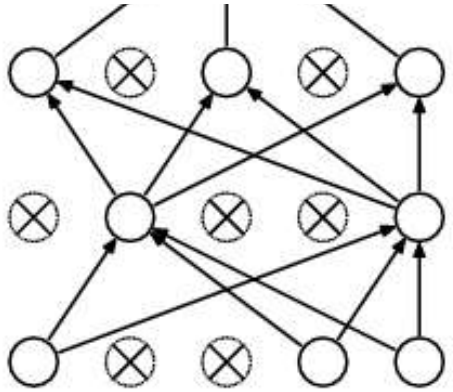
Abstract

Deep neural nets with a large number of parameters are very powerful machine learning systems. However, overfitting is a serious problem in such networks. Large networks are also slow to use, making it difficult to deal with overfitting by combining the predictions of many different large neural nets at test time. Dropout is a technique for addressing this problem. The key idea is to randomly drop units (along with their connections) from the neural network during training. This prevents units from co-adapting too much. During training, dropout samples from an exponential number of different "thinned" networks. At test time, it is easy to approximate the effect of averaging the predictions of all these thinned networks by simply using a single unthinned network that has smaller weights. This significantly reduces overfitting and gives major improvements over other regularization methods. We show that dropout improves the performance of neural networks on supervised learning tasks in vision, speech recognition, document classification and computational biology, obtaining state-of-the-art results on many benchmark data sets.

Keywords: neural networks, regularization, model combination, deep learning



(a) Standard Neural Net



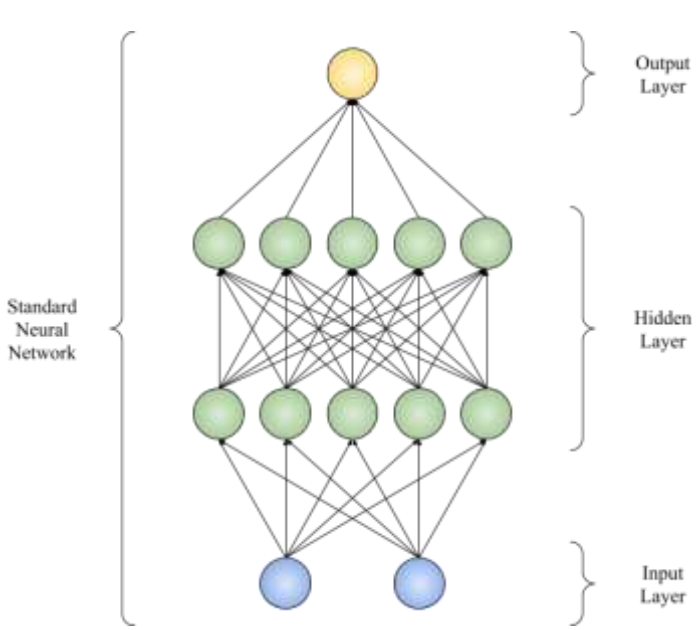
(b) After applying dropout.

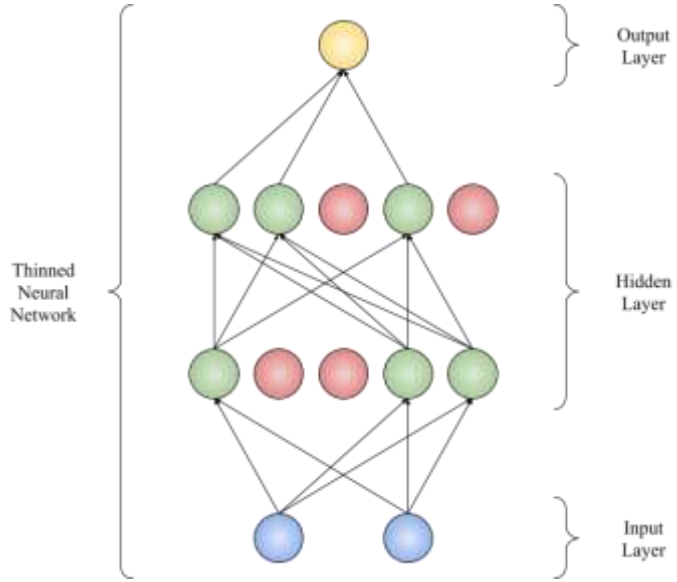
Dropout is a technique that prevents over fitting and provides a way of approximately combining many different neural network architectures.

The term dropout refers to dropping out units (hidden and visible) in a neural network.

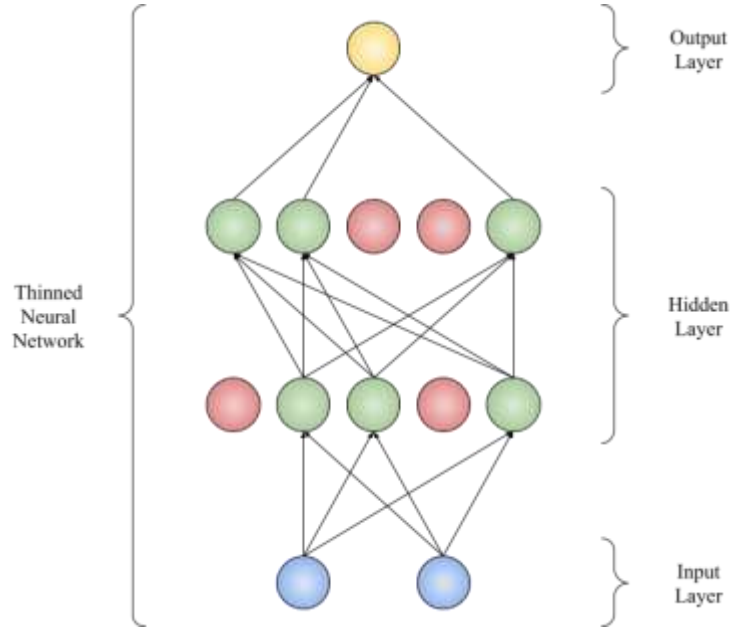
Drop-Out Concept

Dropout is a regularization technique used in deep learning to prevent overfitting by randomly ignoring or "dropping out" a fraction of neurons during the training phase of a neural network



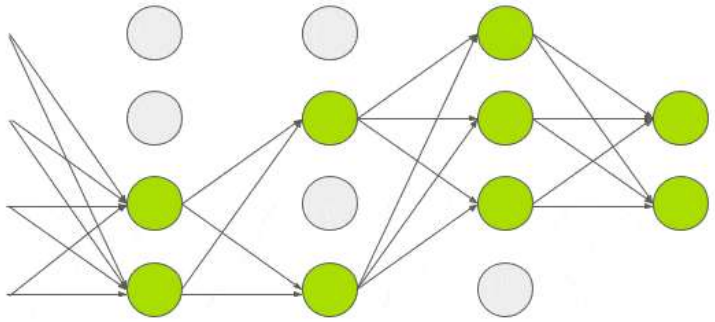


Neural Network with 40% hidden units dropped



Neural Network with 40% hidden units dropped

The primary goal is to prevent the network from becoming overly reliant on specific neurons or feature detectors, a condition known as co-adaptation, which can lead to poor generalization on unseen data.



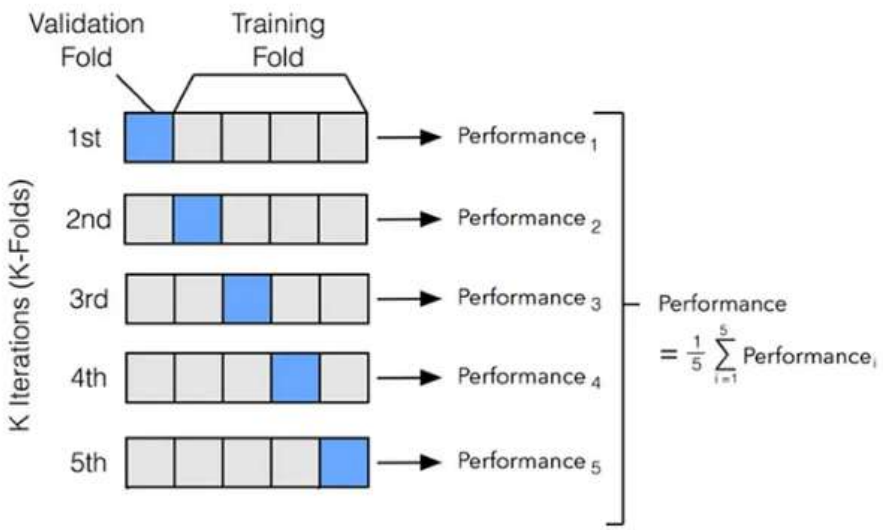
Cross-Validation

Cross-validation is a technique used to evaluate the performance of a machine learning model by training it on different subsets of the data and testing it on the remaining subset

Cross-validation is also known as rotation estimation or out-of-sample testing.

Cross validation is a way to check if a machine learning model is overfitting or underfitting on a given dataset.

K-Fold Cross-Validation



[ ]:
```python
from sklearn.model_selection import KFold
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# create an instance of the model
model = LogisticRegression()

# create an instance of the KFold cross-validator
kf = KFold(n_splits=10)

# create lists to store the accuracy scores for each fold
accuracies = []

# iterate over the splits
for train_index, test_index in kf.split(X):
    # split the data into training and test sets
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    # fit the model on the training data
    model.fit(X_train, y_train)

    # make predictions on the test data
    y_pred = model.predict(X_test)

    # calculate the accuracy of the model
    accuracy = accuracy_score(y_test, y_pred)

    # append the accuracy score to the list
    accuracies.append(accuracy)

# calculate the average accuracy across all folds
average_accuracy = sum(accuracies) / len(accuracies)

# print the average accuracy
print("Average accuracy:", average_accuracy)
```

stratified k-fold cross-validation, the data is first divided into k folds based on the class distribution

Startified K-fold cross validation (K = 5)

class distribution is imbalanced

Dataset of 1000 samples, and we want to train a model to classify the samples as either “M” or “F” class.

The dataset has 600 samples of class “M” and 400 samples of class “F”

This means that each fold will have roughly 120 samples of class “M” and 80 samples of class “F”. This way, the proportion of samples for each class is roughly the same in each fold.

Development of AI - Overview

2015: YOLO (You only look once)

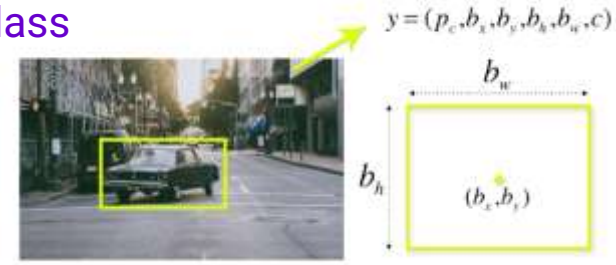
YOLO, a new approach to object detection. It is a single neural network that predicts bounding boxes and class probabilities directly from full images in one evaluation.

Residual blocks

First, the image is divided into various grids. Each grid has a dimension of S x S. The following image shows how an input image is divided into grids.

Bounding box regression

A bounding box is an outline that highlights an object in an image. The attributes: Width, height, Class (Person, car …) and bounding box center



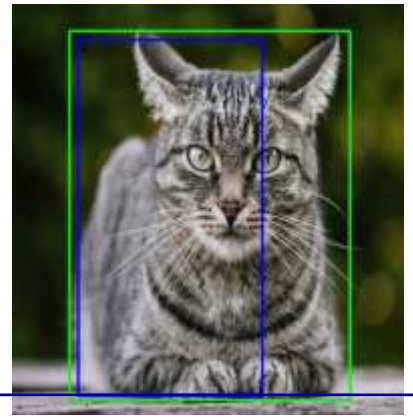
YOLO algorithm works

- Residual blocks
- Bounding box regression
- Intersection Over Union (IOU)

Intersection over union (IOU)

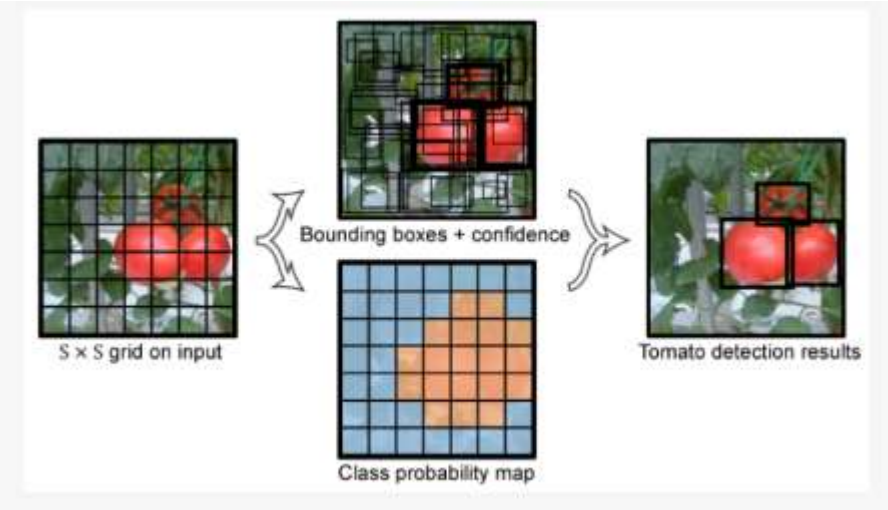
IOU describes how boxes overlap. YOLO uses IOU to provide an output box that surrounds the objects perfectly. Each grid cell is used for predicting the bounding boxes and their confidence scores.

The IOU is equal to 1 (predicted bounding box is the same as the real box).

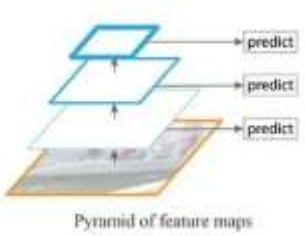


The blue box is the predicted box while the green box is the real box. YOLO ensures that the two bounding boxes are equal.

Development of AI - Overview



FPN = Feature Pyramid Network



Main improvement measures of YOLO network from V1 to V5:

YOLO:
The grid division is responsible for detection, confidence loss;

YOLO V2:
Anchor with K-means added, two-stage training, full convolutional network;

YOLO V3:
Multi-scale detection by using FPN;

YOLO V4:
SPP, MISH activation function, data enhancement Mosaic/Mixup, GIOU(Generalized Intersection over Union) loss function

YOLO V5:
Flexible control of model size, application of Hardswish activation function, and data enhancement.

YOLO V7:
Algorithm is making big waves in the computer vision and machine learning communities. The newest YOLO algorithm surpasses all previous object detection models and YOLO versions in both speed and accuracy. It requires several times cheaper hardware than other neural networks and can be trained much faster on small datasets without any pre-trained weights

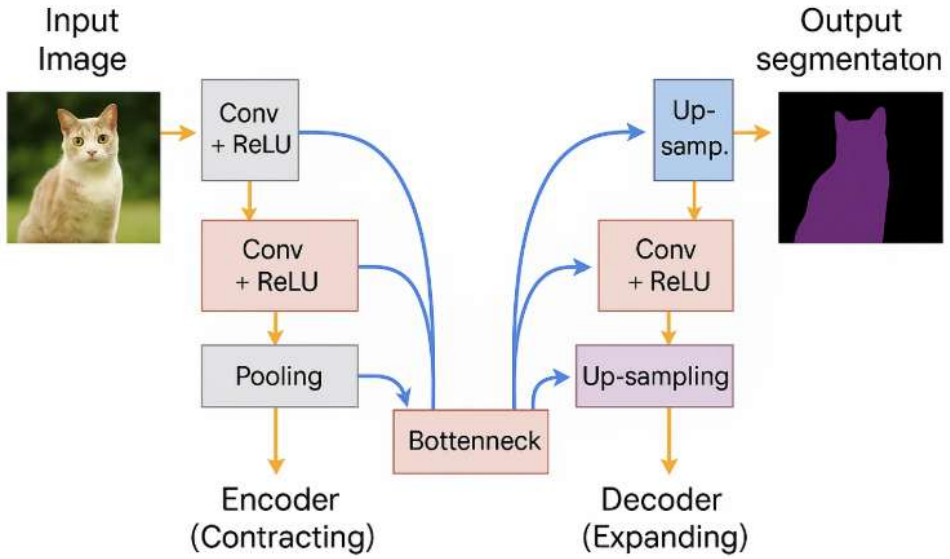
U-Net Architecture

U-Net is a powerful deep learning architecture designed for semantic segmentation

U-Net has **encoder-decoder structure** with **skip connections**, enabling precise localization with fewer training images.

Semantic segmentation, also known as pixel-based classification, is an important task in which we classify each pixel of an image as belonging to a particular class.

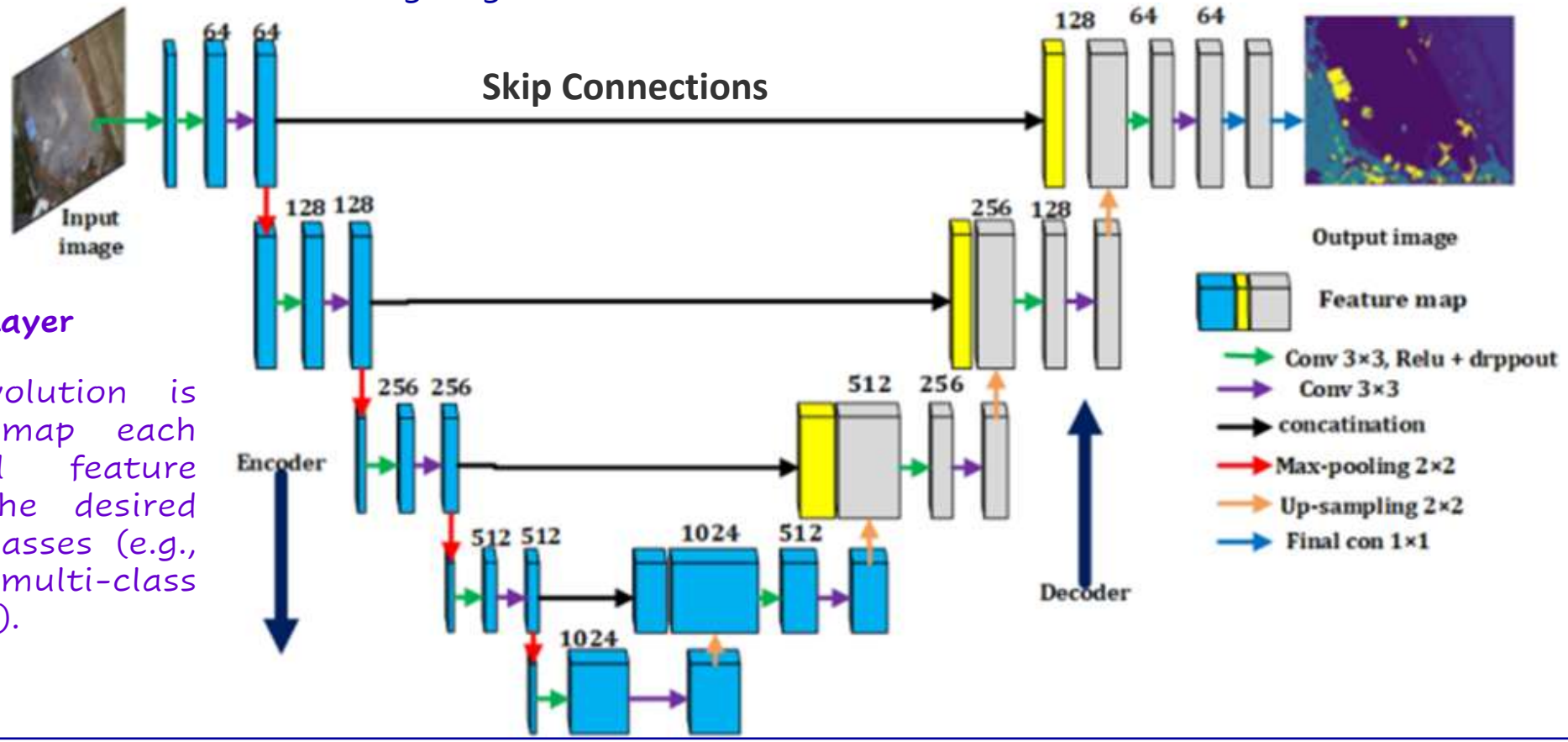
satellite imagery.



Key Components: U-Net Architecture

Recovers spatial information lost during downsampling.

Connect the encoder feature maps to decorder layers allowing large resolution features to be reused.



Final Output Layer

A 1×1 convolution is applied to map each multi-channel feature vector to the desired number of classes (e.g., for binary or multi-class segmentation).

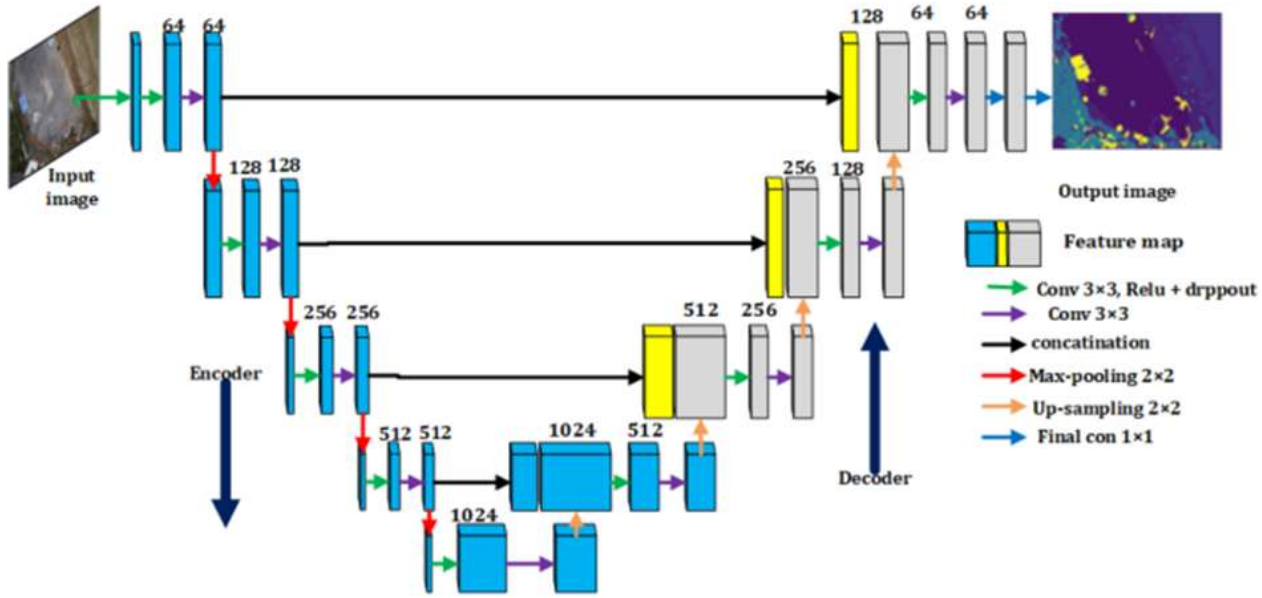
Key Components: U-Net Architecture

Goal:

Capture context and spatial features.

The input image passes through several convolutional layers (Conv + ReLU), each followed by a **max-pooling** operation (downsampling).

This reduces spatial dimensions while increasing the number of feature maps.



Goal: Act as a bridge between the encoder and decoder.

Contains:
Conv. Layers with no pooling. It has highest number of filters.

Deepest part of the network. Where the image representation is most abstract

Goal: Reconstruct spatial dimensions and locate objects more precisely

Each step includes an **upsampling** that increases the resolution.

The output is then concatenated with corresponding feature maps from the encoder (from the same resolution level) via **skip connections**.

Followed by standard convolution layers.